Day 23: Logs and Regular Expressions

1. What are system and web logs?

A **log** is a text file where software records events as they happen, one line per event. Logs answer the questions: *what happened, when, and who or what caused it?*

System logs record what the operating system and its services do:

- Logins and failed login attempts (e.g., /var/log/auth.log on Linux)
- Service starts and stops, crashes, hardware errors (/var/log/syslog)

Web logs record every request made to a web server (Apache, Nginx):

- Which IP address visited
- Which page or file was requested
- When, and whether it succeeded (status code 200, 404, 500, etc.)

Logs are used for debugging, security monitoring (spotting brute-force attacks), and traffic analysis. Since they are huge and repetitive, we use Python to analyze them automatically.

---

2. Reading a sample log file

A typical web server log line (Apache "Common Log Format"):

192.168.1.10 - - [28/Oct/2026:10:15:32 +0530] "GET /index.html HTTP/1.1" 200 5120

Part	Meaning
192.168.1.10	Client IP address
[28/Oct/2026:10:15:32 +0530]	Timestamp
"GET /index.html HTTP/1.1"	Request method, page, protocol
200	Status code (success)
5120	Response size in bytes

Suppose you save these lines in access.log:

192.168.1.10 - - [28/Oct/2026:10:15:32 +0530] "GET /index.html HTTP/1.1" 200 5120

203.0.113.45 - - [28/Oct/2026:10:16:01 +0530] "GET /login HTTP/1.1" 200 1024

203.0.113.45 - - [28/Oct/2026:10:16:05 +0530] "POST /login HTTP/1.1" 401 512

10.0.0.7 - - [28/Oct/2026:10:17:44 +0530] "GET /missing.html HTTP/1.1" 404 300

Reading it in Python:

python

```
with open("access.log", "r") as file:
    for line in file:
        print(line.strip())
```

Using with closes the file automatically. Looping line by line is memory-efficient, which matters for logs that are gigabytes in size.

Plain string methods like split() can extract fields, but they break when the format varies slightly. That's where regular expressions come in.

---

3. Introduction to the re module

A **regular expression (regex)** is a pattern that describes text. Python's built-in re module lets you search for those patterns.

python

```
import re
```

Common functions

Function	What it does
re.search(pattern, text)	Finds the **first** match anywhere in the text
re.match(pattern, text)	Matches only at the **start** of the text
re.findall(pattern, text)	Returns a **list of all** matches
re.sub(pattern, repl, text)	Replaces matches with something else
re.compile(pattern)	Pre-builds a pattern for reuse (faster in loops)

Common pattern symbols

Symbol	Meaning
\d	Any digit (0-9)
\w	Letter, digit, or underscore
\s	Whitespace
.	Any character (except newline)
+	One or more of the previous item
*	Zero or more
{1,3}	Between 1 and 3 repetitions

Symbol Meaning

\. A literal dot (since . alone is a wildcard)

[A-Z] Any one character in the set

( ) Capture group

Always write patterns as **raw strings** (r"...") so Python doesn't misread backslashes.

python

```
text = "Error 404 on page 12"
print(re.findall(r"\d+", text))   # ['404', '12']
```

---

4. Finding IP addresses

An IPv4 address looks like 203.0.113.45: four numbers (each 1-3 digits) separated by dots.

Pattern:

python

```
r"\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}"
```

Breaking it down: \d{1,3} is 1-3 digits, \. is a literal dot, repeated four times.

Example: extract all IPs from the log

python

```
import re

ip_pattern = re.compile(r"\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}")

ips = []
with open("access.log", "r") as file:
    for line in file:
        match = ip_pattern.search(line)   # first IP on the line
        if match:
            ips.append(match.group())

print(ips)
```

Output:

['192.168.1.10', '203.0.113.45', '203.0.113.45', '10.0.0.7']

Counting requests per IP

python

```
from collections import Counter


counts = Counter(ips)

for ip, n in counts.most_common():

    print(ip, n)
```

Output:

203.0.113.45 2

192.168.1.10 1

10.0.0.7 1

This is the basis of spotting suspicious activity: an IP with hundreds of failed logins (status 401) stands out immediately.

Extracting several fields at once with groups

python

```
pattern = re.compile(r'^(\S+) .*?"(\w+) (\S+) .*?" (\d{3})')


with open("access.log") as file:

    for line in file:

        m = pattern.match(line)

        if m:

            ip, method, path, status = m.groups()

            print(ip, method, path, status)
```

A note on accuracy

The simple pattern also matches invalid values like 999.999.999.999. For learning and most log work that's fine, since logs contain real IPs. If you need strict validation, use Python's ipaddress module:

python

```
import ipaddress

ipaddress.ip_address("203.0.113.45")   # valid; raises ValueError otherwise
```

Quick practice

1. Count how many lines have status 404.

2. List all IPs that received a 401.
3. Find the IP that made the most requests.

Want me to walk through solutions to these, or generate a bigger sample log file to practice on?